



DOCUMENTO DE DESCRIPCIÓN DEL SISTEMA · V1.0

Banco de Hojas de Vida

Facultad de Ciencias Económicas
Universidad del Atlántico

CLIENTE

FCE – Universidad del Atlántico

VERSIÓN

1.0.0

FECHA

Junio 2026

CLASIFICACIÓN

Uso interno institucional

Tabla de Contenido

1. Descripción General	3
1.1 Propósito	3
1.2 Alcance	3
1.3 Contexto institucional	3
2. Funcionalidades del Sistema	4
2.1 Módulo de Autenticación	4
2.2 Módulo de Hoja de Vida	4
2.3 Módulo de Vacantes y Postulaciones	4
2.4 Módulo de Empresas	5
2.5 Módulo Administrativo	5
2.6 Sistema de Roles Múltiples	5
3. Arquitectura del Sistema	6
3.1 Patrón arquitectónico	6
3.2 Capas de la aplicación	6
3.3 Flujo de una solicitud HTTP	6
4. Stack Tecnológico	7
5. Modelo de Datos	8
5.1 Entidades principales	8
5.2 Relaciones entre entidades	8
6. Seguridad y Control de Acceso	9
7. Gestión de Archivos	9
8. Estructura del Proyecto	10
9. Requisitos de Infraestructura	10
10. Identidad Visual	11



SECCIÓN 1

Descripción General

1.1 Propósito

El **Banco de Hojas de Vida FCE** es una plataforma web de gestión de talento humano desarrollada para la **Facultad de Ciencias Económicas de la Universidad del Atlántico**. Su objetivo es centralizar y digitalizar el proceso de intermediación laboral entre estudiantes y egresados de la facultad y las empresas aliadas.

El sistema permite a los candidatos construir su perfil profesional digital, explorar ofertas laborales y hacer seguimiento a sus postulaciones; mientras que el Líder de Prácticas gestiona las empresas y vacantes, y el Administrador controla los accesos y la operación global del sistema.

1.2 Alcance



Incluye

Registro de usuarios via Google OAuth institucional · Creación y edición de hoja de vida digital · Carga de archivos (foto, CV, certificados) · Publicación y consulta de vacantes · Sistema de postulaciones · Gestión de empresas aliadas · Panel administrativo con estadísticas · Sistema de roles múltiples por usuario · Mensajes en respuesta a candidaturas



No incluye

Pagos en línea · App móvil nativa · Integración con sistemas externos de RRHH · Sistema de video-entrevistas · Notificaciones por correo electrónico · Portal de autoservicio para empresas · Reportes avanzados de analítica · Inteligencia artificial para matching de candidatos

1.3 Contexto institucional

La Universidad del Atlántico, con sede en Barranquilla, Colombia, es una institución pública de educación superior acreditada de alta calidad. La Facultad de Ciencias Económicas requería un canal propio para conectar a su comunidad académica con el mercado laboral de la región Caribe, diferenciado del sistema central universitario.



RESTRICCIÓN DE ACCESO

El sistema solo acepta cuentas de Google con correo institucional @uniatlantico.edu.co y todos sus subdominios (@mail.uniatlantico.edu.co , @est.uniatlantico.edu.co , etc.). Cuentas personales son rechazadas en el momento del login.



SECCIÓN 2

Funcionalidades del Sistema

2.1 Módulo de Autenticación

- Inicio de sesión exclusivo mediante **Google OAuth 2.0**.
- Validación del dominio del correo electrónico en el servidor (no en el cliente).
- Creación automática del usuario en la primera autenticación exitosa.
- Gestión de sesiones seguras con `express-session` y almacenamiento en PostgreSQL.
- Cierre de sesión con destrucción de la sesión en el servidor.

2.2 Módulo de Hoja de Vida

Cada usuario dispone de una hoja de vida digital estructurada con las siguientes secciones:

Sección	Campos principales
Perfil general	Resumen profesional, teléfono, ciudad, LinkedIn, portafolio web
Foto de perfil	Imagen JPG/PNG/WEBP (máx. 5 MB) subida al servidor
CV adjunto	Archivo PDF/DOC/DOCX descargable (máx. 10 MB)
Educación	Institución, título, campo de estudio, año inicio/fin, en curso
Experiencia laboral	Empresa, cargo, descripción, fecha inicio/fin, trabajo actual
Habilidades	Nombre de la habilidad y nivel
Idiomas	Nombre del idioma y nivel de dominio
Documentos	Certificados, diplomas, referencias (PDF/JPG/PNG, máx. 10 MB c/u)

2.3 Módulo de Vacantes y Postulaciones

- Listado público (para usuarios autenticados) de vacantes activas con datos de empresa, área y modalidad.
- Vista de detalle de la vacante con requisitos, descripción y botón de postulación.
- Postulación con carta de presentación opcional. Una sola postulación por vacante por usuario.
- Panel personal *Mis Postulaciones* con estado actual (Pendiente / Aceptado / Rechazado).
- Mensaje de retroalimentación del Líder visible en la candidatura del estudiante.
- Estados de vacante: **Abierta**, **Cerrada**, **Borrador**.

2.4 Módulo de Empresas

- Registro de empresas aliadas: nombre, NIT, sector, descripción, sitio web y logo.

- Las empresas **no tienen acceso directo al sistema**; son gestionadas por el Líder de Prácticas.
- Cada vacante está asociada a una empresa del catálogo.
- Todas las empresas registradas están disponibles al crear cualquier vacante.

2.5 Módulo Administrativo

- Dashboard con métricas: usuarios, hojas de vida, vacantes activas, postulaciones totales.
- Listas recientes de usuarios y postulaciones para seguimiento.
- Gestión de usuarios: búsqueda, filtro por rol, asignación de múltiples roles, eliminación.
- Acceso a la hoja de vida de cualquier usuario del sistema.
- Moderación global de vacantes: visualización y eliminación de cualquier vacante.

2.6 Sistema de Roles Múltiples

Un usuario puede tener simultáneamente uno o más roles asignados. El rol activo en sesión determina la vista y el menú de navegación. El Administrador siempre mantiene acceso completo independientemente del rol activo seleccionado.



SECCIÓN 3

Arquitectura del Sistema

3.1 Patrón arquitectónico

El sistema sigue una arquitectura **monolítica de servidor con renderizado en el servidor (SSR)**, implementada como una aplicación **MVC** (Modelo – Vista – Controlador) sobre Node.js.

DECISIÓN DE DISEÑO

Se eligió la arquitectura monolítica SSR (en lugar de SPA + API REST) por simplicidad de despliegue, menor latencia en la carga inicial, mejor indexabilidad y reducción de la superficie de ataque. Para el volumen de usuarios esperado (cientos, no miles simultáneos), este modelo es óptimo.

3.2 Capas de la aplicación

1

Vista (Templates EJS)

Plantillas EJS renderizadas en el servidor. Incluyen partials reutilizables (navbar, footer). Los estilos se generan con Tailwind CSS compilado estáticamente.

2

Rutas (Express Router)

Define los endpoints HTTP. Aplica middlewares de autenticación y autorización por rol antes de llamar al controlador correspondiente.

3

Controladores

Lógica de negocio: valida entradas con Zod, coordina operaciones de la base de datos vía Prisma, gestiona archivos con Multer y renderiza la vista con datos.

4

ORM / Acceso a datos (Prisma)

Prisma Client genera queries tipadas hacia PostgreSQL. El esquema define modelos, relaciones y enums. Las migraciones se gestionan con SQL directo cuando los cambios son complejos.

5

Base de datos (PostgreSQL)

Almacena todos los datos del sistema. Las sesiones de usuario también se persisten en PostgreSQL mediante connect-pg-simple.

3.3 Flujo de una solicitud HTTP

1

Navegador — El usuario hace una acción (clic, envío de formulario). El navegador envía una petición HTTP al servidor.

2

Express + Middlewares — Se verifica la sesión (`express-session`), se restaura el usuario de la BD (`passport.session()`), se comprueba el rol activo.

- 3 **Router** — La URL coincide con una ruta. Si hay middleware de autorización (`requireRole`), se evalúa el rol antes de continuar.
- 4 **Controlador** — Valida el body con Zod, llama a Prisma para leer/escribir datos, procesa archivos si los hay (Multer).
- 5 **Vista EJS** — El controlador llama a `res.render()` con los datos. EJS compila el HTML y lo envía al cliente como respuesta.
- 6 **Navegador** — Recibe el HTML completo y lo muestra. Para mutaciones (POST) se usa el patrón PRG (Post–Redirect–Get) para evitar reenvíos accidentales.

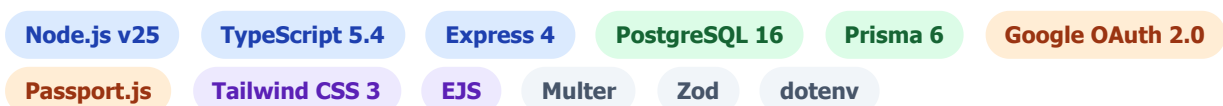


SECCIÓN 4

Stack Tecnológico

Categoría	Tecnología	Versión	Propósito
Runtime	Node.js	v25+	Entorno de ejecución del servidor
Lenguaje	TypeScript	5.4	Tipado estático, autocompletado, prevención de errores en tiempo de desarrollo
Framework web	Express.js	4.19	Enrutamiento HTTP, middlewares, gestión de peticiones/respuestas
Motor de plantillas	EJS	3.1	Renderizado HTML en el servidor con datos dinámicos
Estilos	Tailwind CSS	3.4	Utility-first CSS, compilado como archivo estático
ORM	Prisma	6.19	Acceso a base de datos con esquema tipado y migraciones
Base de datos	PostgreSQL	16	Almacenamiento persistente de todos los datos del sistema
Autenticación	Passport.js + Google OAuth 2.0	0.7	Login delegado a Google, validación de dominio institucional
Sesiones	express-session + connect-pg-simple	1.18 / 9.0	Sesiones persistentes almacenadas en PostgreSQL
Carga de archivos	Multer	1.4 LTS	Manejo de multipart/form-data, filtros de tipo y tamaño
Validación	Zod	3.23	Validación y parseo de datos de formularios con tipos TypeScript
Variables de entorno	dotenv	16.4	Carga de configuración desde archivo <code>.env</code>
Dev: ejecución	tsx	4.15	Ejecución directa de TypeScript sin compilar (modo desarrollo)
Dev: PDF	Puppeteer	25.1	Generación de documentos PDF desde HTML (scripts internos)

Etiquetas por categoría



⚠️ **NOTA DE COMPATIBILIDAD**

Node.js v25 presenta incompatibilidades con algunas herramientas del ecosistema (`concurrently` , `prisma studio`). La compilación de Tailwind CSS se ejecuta como comando separado (`npm run css`). Los cambios complejos de esquema en base de datos se realizan con SQL directo seguido de `prisma generate` .



SECCIÓN 5

Modelo de Datos

5.1 Entidades principales

User (users)

id	<code>String</code> (cuid) PK
email	<code>String</code> único, correo institucional
name	<code>String</code> nombre completo de Google
photo	<code>String?</code> URL avatar de Google
roles	<code>Role[]</code> array: STUDENT LIDER ADMIN
googleId	<code>String?</code> ID único de Google

CurriculumVitae

userId	<code>String</code> FK → User (1:1)
summary	<code>String?</code> resumen profesional
phone	<code>String?</code> teléfono de contacto
city	<code>String?</code> ciudad de residencia
cvFileUrl	<code>String?</code> ruta del PDF/Word subido
photoUrl	<code>String?</code> foto de perfil subida

Company (companies)

userId	<code>String</code> FK → User (quién la registró)
name	<code>String</code> razón social
nit	<code>String?</code> NIT de la empresa
sector	<code>String?</code> sector económico
logoUrl	<code>String?</code> ruta del logo
website	<code>String?</code> sitio web

Vacancy (vacancies)

companyId	<code>String</code> FK → Company
title	<code>String</code> título del cargo
area	<code>String</code> área de la vacante
type	<code>String</code> modalidad (TC, MT, práctica...)
status	<code>VacancyStatus</code> OPEN CLOSED DRAFT
salary	<code>String?</code> rango salarial (opcional)

Application (applications)

userId	<code>String</code> FK → User	vacancyId	<code>String</code> FK → Vacancy · unique(userId+vacancyId)
status	<code>ApplicationStatus</code> PENDING REVIEWED ACCEPTED REJECTED	coverLetter	<code>String?</code> carta de presentación del candidato
message	<code>String?</code> mensaje del Líder al candidato	appliedAt	<code>DateTime</code> fecha de postulación

5.2 Relaciones entre entidades

Relación	Tipo	Detalle
User → CurriculumVitae	1 : 0..1	Cada usuario tiene como máximo una HV. Cascade delete.
User → Company	1 : 0..1	Un usuario Líder puede crear una empresa. Cascade delete.
Company → Vacancy	1 : N	Una empresa tiene múltiples vacantes. Cascade delete.
CurriculumVitae → Education/Experience/Skill/Language/Document	1 : N	La HV tiene múltiples entradas por sección. Cascade delete.
User → Application	1 : N	Un usuario puede postularse a varias vacantes.
Vacancy → Application	1 : N	Una vacante tiene múltiples candidatos.
User + Vacancy → Application	Unique	Restricción: máximo una postulación por usuario por vacante.



SECCIÓN 6

Seguridad y Control de Acceso

Autenticación

- **Sin contraseñas propias:** toda la autenticación se delega a Google OAuth 2.0, eliminando los riesgos de almacenamiento de contraseñas.
- **Validación de dominio:** la función `isInstitutionalEmail()` verifica el dominio del correo en el callback de Google antes de crear o autenticar al usuario.
- **Sesiones firmadas:** las sesiones usan un `SESSION_SECRET` configurado en variables de entorno. Se almacenan en PostgreSQL (no en memoria) para sobrevivir reinicios.

Autorización por roles

- El middleware `requireRole(...roles)` protege cada ruta con los roles permitidos.
- El rol **ADMIN** siempre tiene acceso completo, independientemente del rol activo en sesión.
- El rol activo en sesión (`req.session.activeRole`) controla la vista del menú, no los permisos reales del servidor.
- Todas las verificaciones de permisos ocurren en el servidor, nunca solo en el cliente.

Protecciones adicionales

Amenaza	Mitigación implementada
SQL Injection	Prisma usa consultas parametrizadas. No hay SQL dinámico concatenado.
XSS	EJS escapa por defecto las variables con <code><%= %></code> . El HTML sin escapar (<code><%-></code>) se usa solo en partials de confianza.
Subida de archivos maliciosos	Multer filtra por <code>mimetype</code> y extensión. Hay límite de tamaño por tipo de archivo.
Acceso no autenticado	El middleware <code>requireAuth</code> redirige a <code>/auth/login</code> si no hay sesión activa.
Escalada de privilegios	Los roles solo los asigna el Administrador. No hay endpoint público para auto-asignación.
Fijación de sesión	Passport regenera la sesión tras el login (<code>req.logIn + session.regenerate</code>).

⚠ **PENDIENTE PARA PRODUCCIÓN**

Se recomienda agregar en producción: cabeceras de seguridad HTTP (Helmet.js), protección CSRF en formularios POST, límite de tasa en el endpoint de OAuth, y HTTPS obligatorio mediante un proxy inverso (nginx).



SECCIÓN 7

Gestión de Archivos

El sistema gestiona la carga de archivos mediante **Multer**. Los archivos se almacenan en el servidor de forma local, en el directorio configurado por la variable de entorno `UPLOAD_DIR`.

Tipos de archivo por módulo

Campo	Módulo	Formatos permitidos	Límite	Destino
Foto de perfil de HV	Hoja de Vida	image/jpeg, image/png, image/webp	5 MB	uploads/photos/
CV adjunto	Hoja de Vida	application/pdf, .doc, .docx	10 MB	uploads/cv/
Documentos / certificados	Hoja de Vida	PDF, JPG, PNG	10 MB c/u	uploads/documents/
Logo de empresa	Empresas	image/jpeg, image/png	5 MB	uploads/logos/

Generación de nombres de archivo

Los nombres se generan con un UUID v4 + extensión original para evitar colisiones y ocultar nombres originales. El path relativo se guarda en la base de datos y se sirve como archivo estático bajo la ruta `/uploads/*`.

RECOMENDACIÓN PARA ESCALA

En una instalación de producción con alto volumen de usuarios, se recomienda migrar el almacenamiento de archivos a un servicio de objetos externo (AWS S3, MinIO, Cloudflare R2) para desacoplar el almacenamiento del servidor de aplicación y facilitar la escalabilidad horizontal.

Archivos estáticos públicos

Además de los uploads, el servidor sirve los archivos estáticos de la carpeta `src/public/`:

- `src/public/css/styles.css` — CSS compilado de Tailwind
- `src/public/js/` — Scripts de cliente (mínimos)



SECCIÓN 8

Estructura del Proyecto

```
bancohv-fce/ ├── prisma/ | └─ schema.prisma # Esquema de base de datos (Prisma ORM)
└─ scripts/ | └─ generate-manual.mjs # Genera Manual de Usuario en PDF | └─
generate-descripcion.mjs # Genera este documento en PDF └─ src/ | └─ config/ | |
└─ passport.ts # Estrategia Google OAuth 2.0 | └─ controllers/ | | └─
admin.controller.ts # Dashboard, usuarios, roles, moderación | | └─
cv.controller.ts # CRUD hoja de vida y secciones | | └─ vacancy.controller.ts #
Vacantes, empresas, postulaciones | └─ middlewares/ | | └─ auth.middleware.ts #
requireAuth, requireRole, getActiveRole | └─ routes/ | | └─ admin.routes.ts #
/admin/* | | └─ auth.routes.ts # /auth/google, /auth/callback, /auth/logout | | └─
company.routes.ts # /company/* (protegido: LIDER|ADMIN) | | └─ cv.routes.ts # /cv/*
| | └─ role.routes.ts # /role/switch (cambio de rol activo) | | └─
vacancy.routes.ts # /vacancies/* | | └─ index.ts # Composición de rutas en app |
└─ utils/ | | └─ auth.ts # isInstitutionalEmail(), env validation | └─ public/ |
└─ css/styles.css # Tailwind compilado | └─ views/ | | └─ partials/ #
navbar.ejs, footer.ejs | | └─ pages/ # auth/, cv/, dashboard/, vacancies/,
company/, admin/ | └─ app.ts # Punto de entrada: Express, middlewares, rutas └─
uploads/ # Archivos subidos por usuarios (fotos, CVs, docs) └─ .env # Variables de
entorno (no versionado) └─ tailwind.config.js # Colores institucionales UA 2024 └─
tsconfig.json └─ package.json
```



SECCIÓN 9

Requisitos de Infraestructura

Requisitos mínimos del servidor

Componente	Mínimo	Recomendado
CPU	1 vCPU	2 vCPU
RAM	1 GB	2 GB
Disco	20 GB	50 GB+ (para archivos de usuarios)
Node.js	v20 LTS	v22 LTS
PostgreSQL	14	16
SO	Ubuntu 22.04	Ubuntu 24.04 LTS

Variables de entorno requeridas

Variable	Descripción
DATABASE_URL	Cadena de conexión a PostgreSQL (postgresql://user:pass@host:5432/db)
GOOGLE_CLIENT_ID	Client ID de la aplicación registrada en Google Cloud Console
GOOGLE_CLIENT_SECRET	Client Secret de la aplicación de Google
GOOGLE_CALLBACK_URL	URL de retorno OAuth (https://dominio.com/auth/google/callback)
SESSION_SECRET	Clave secreta aleatoria para firmar sesiones (mín. 32 caracteres)
UPLOAD_DIR	Ruta local para almacenar archivos subidos (ej. uploads)
PORT	Puerto del servidor HTTP (por defecto: 3000)

Comandos de despliegue

```
# 1. Instalar dependencias
npm install

# 2. Generar cliente Prisma
npx prisma generate
```


3. Aplicar migraciones

```
npx prisma migrate deploy
```

4. Compilar CSS

```
npm run css
```

5. Compilar TypeScript

```
npx tsc
```

6. Iniciar en producción

```
node dist/app.js
```

✅ DESARROLLO LOCAL

Para desarrollo: `npm run dev (tsx watch)` + `npm run css` (en terminal separada para compilar Tailwind).



SECCIÓN 10

Identidad Visual

La interfaz aplica la identidad visual oficial de la Universidad del Atlántico según el **Manual de Identidad Visual 2024** (Resolución Rectoral 001536 del 17 de junio de 2024).

Paleta de colores institucionales

				
Azul Primario #143163 RGB 20·49·99	Naranja Primario #D85819 RGB 216·88·25	Azul Medio #1D71B8 RGB 29·113·184	Naranja Dorado #FF9912 RGB 255·153·18	Amarillo Dorado #F9B233 RGB 249·178·51
				
Gris #706F6F RGB 112·111·111				

Tipografía

Uso	Fuente	Variante
Interfaz web (body, menús, formularios)	Tahoma	Regular / Bold
Alternativa elegante (documentos)	Candara	Regular
Material promocional (si disponible)	Radikal	Bold / Regular / Thin
Código y rutas en documentación	Courier New	Regular (monoespaciada)

Aplicación en la interfaz

- La **barra de navegación** usa fondo #143163 (azul primario) con texto blanco.
- Los **encabezados** de sección usan #143163 ; los acentos y botones de acción primaria usan #D85819 .
- Las **tarjetas del dashboard** tienen bordes laterales de color para distinguir módulos.
- El **fondo de la pantalla de login** es azul marino primario con el ícono UA en naranja.
- El **botón de cerrar sesión** usa el naranja primario sobre azul, reforzando la identidad en el punto de salida.

Banco de Hojas de Vida FCE · v1.0.0

Facultad de Ciencias Económicas · Universidad del Atlántico

Barranquilla, Colombia · Junio 2026

www.uniatlantico.edu.co